1. Python is an important versatile programming language that offers a wide range of modules for developers. One of the standout modules for making HTTP requests is the requests module. It's known for its straightforwardness, user-friendliness, and flexibility.

This module accommodates various HTTP request types such as GET, POST, PUT, DELETE, among others. Below is a comprehensive list of all the request methods available through the requests module:

- requests.get(url, params=None
- requests.post(url, data=None, json=None,
- requests.put(url, data=None,
- requests.delete(url,
- requests.head(url,
- requests.options(url,
- requests.patch(url, data=None,
- requests.request(method, url,

To send a request, we first need to import the requests module and use its methods. For instance, to send a GET request, we can use the get() method as follows:

```
# Import the requests module
import requests
# Make a get() request
response = requests.get('<https://jsonplaceholder.typicode.com/posts>')
# Print the status / content of the request
print(response.status_code)
print(response.json())
```

This code sends a GET request to a sample REST API and prints the response status code and JSON data. The response object returned by get() contains attributes and methods for dealing with the response data, such as the following:, such as status_code, text, json().

2. JSON is a lightweight data interchange format that is easy for humans to read and write and easy for machines to parse and generate. It is built on two structures: a collection of name/value pairs and an ordered list of values. JSON's simplicity and compatibility with most programming languages make it a popular choice for web APIs and configuration files.

Advantages

- **Lightweight and Compact:** JSON uses a minimalist key-value syntax without opening and closing tags, which results in smaller file sizes and lower bandwidth consumption.
- **Fast Parsing Speed:** Machines and web browsers can parse and generate JSON data much faster due to its simple structural overhead.
- **Native Array Support:** JSON natively supports ordered lists and arrays, matching modern programming language data structures easily.

- **High Readability and Simplicity:** Its syntax closely resembles common object notation, making it intuitive for humans to read, write, and debug.

Disadvantages

- **No Official Comment Support:** Standard JSON does not allow comments (such as // or /* */), making inline documentation difficult.
- **Lacks Schema and Validation:** JSON lacks a built-in schema validation mechanism on its own, requiring external tools (like JSON Schema) to enforce strict rules.
- **No Support for Namespaces:** JSON does not support namespaces, which can lead to naming collisions in large or complex data systems.
- **Limited Complex Data Types:** JSON does not natively support advanced types like dates, custom attributes, or binary data, which must be converted to strings or numbers.

XML: The Versatile Markup Language

XML is a flexible markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable. It is highly extensible and can be used to represent complex data structures. XML is widely used in various applications, including web services, configuration files, and document storage.

Advantages

- **Strict Schema Validation:** XML supports comprehensive schema standards like XSD (XML Schema Definition) and DTD to strictly validate data structures.
- **Support for Namespaces:** XML allows custom namespaces to separate and organize different vocabularies or data categories within the same document.
- **Rich Metadata and Attributes:** XML elements can hold custom attributes and metadata directly within the tags, fitting complex documents and hierarchical relationships.
- **Built-in Comment Support:** XML natively supports standard HTML-style comments (<!-- comment -->) for documenting code or data feeds.

Disadvantages

- **Verbose and Bulky Syntax:** The requirement for matching opening and closing tags makes XML files significantly larger, consuming more storage and bandwidth.
- **Slower Parsing Performance:** Processing and traversing the XML DOM (Document Object Model) requires more CPU resources and takes longer than JSON.
- **No Native Array Type:** XML does not have a native array type, requiring developers to use repeated sibling elements to represent lists.
- **Harder to Read Manually:** The heavy usage of angle brackets, tag names, and entity references makes large XML files cluttered and difficult for humans to scan quickly.

3. **A RESTful API** (Representational State Transfer Application Programming Interface) is an architectural style for designing networked software. It lets two systems—such as a mobile app and a web server—communicate over the internet using standard HTTP protocols.

How It Works

- **The Request:** A client (like a web browser or mobile app) sends an HTTP request to a server endpoint (a specific URL).
- **The Method:** The request uses an HTTP verb that maps to a CRUD operation:
- GET: Retrieve data
- POST: Create new data
- PUT / PATCH: Update existing data
- DELETE: Remove data
- **The Processing:** The server receives the request, handles the logic, and is **stateless** (it stores no session info about past requests).
- **The Response:** The server sends back an HTTP status code (such as 200 OK or 404 Not Found) along with a payload, usually formatted as JSON.

What It Is Used For

- **Web and Mobile Apps:** Connecting a frontend interface to backend databases.
- **Third-Party Integrations:** Allowing external services to safely share data (such as payment gateways, maps, or social media logins).
- **Microservices:** Enabling independent internal services in a larger application to talk to one another.
- **Internet of Things (IoT):** Sending telemetry and status updates from physical hardware to cloud servers.

Advantages of RESTful APIs

- **Language and Platform Independence:** Clients and servers can be built in completely different programming languages (e.g., Python backend with a React frontend).
- **Scalability:** Statelessness ensures each request is separate, making it easy to scale servers across load balancers.
- **Cacheability:** Responses can be cached by browsers or proxy servers to cut down server load and speed up performance.
- **Standardized Interface:** Using clear HTTP methods and status codes makes development, testing, and debugging predictable.

Disadvantages of RESTful APIs

- **Over-fetching and Under-fetching:** Fixed endpoint structures often return too much unneeded data or force the client to make multiple requests to get everything it needs.
- **Multiple Request Overhead (Chattiness):** Complex views or relational data may require chaining several separate API calls to assemble a single screen.
- **Lack of Built-in Schema Enforcement:** Unlike strict query languages, REST does not natively enforce a strongly-typed data contract out of the box.

- **Versioning Complexity:** Breaking changes require maintaining multiple API versions (e.g., /v1/ vs /v2/) in parallel.